

Architecture and Micro-architecture

COMS10015

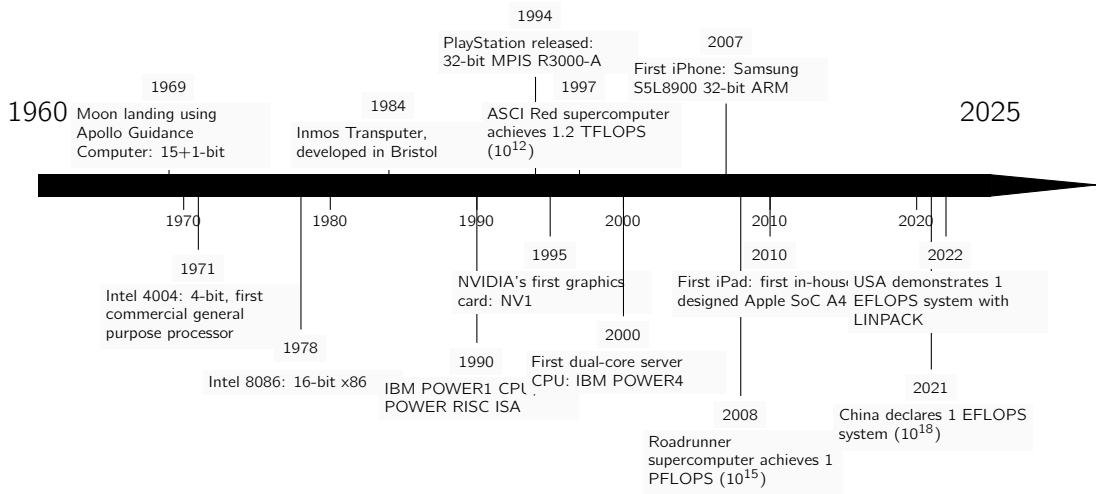
Dr Tom Deakin

University of Bristol

Week 13

Processor timeline

Some interesting processors and supercomputing milestones



Instruction Set Architecture (ISA)

Instruction Set Architecture

- ▶ Intuitivietly, the ISA/architecture defines the “language” the processor understands.
- ▶ Interface between hardware and software.
- ▶ Describes how software controls the hardware.

An ISA defines (as a minimum):

- ▶ Word size.
- ▶ State (e.g., registers, memory).
- ▶ List of instructions and how they interact with the state.
 - ▶ What they do and how they are represented.
- ▶ Execution model.

Based on slides from Dan Page

ISA design

Designing an ISA is *creative*.

- ▶ How easy is it to implement?
- ▶ How usable is it?
- ▶ How efficient is it?
- ▶ How useful is it?
- ▶ Is it orthongonal (can all instructions use all addressing modes)?
- ▶ Is there market demand for it?
- ▶ Does someone already own the IP/patent?

But can do a lot in the implementation to work around differences in ISA.

e.g., Weaver and McIntosh-Smith, An Empirical Comparison of the RISC-V and AArch64 Instruction Sets, 2023,

<https://doi.org/10.1145/3624062.3624233>

Based on slides from Dan Page

ISA state: registers

Usually two main components of Architectural state: *registers* and *memory*.

Often two kinds of registers:

General-purpose register Software-controlled purpose and accessible in typical instructions. Collection is called a *register file*.

Special-purpose register Has a special role, and only accessible by using special instructions.

Examples of special-purpose registers include:

program counter address of next instruction to be executed

instruction register currently executing instruction

status register information about state (and can impact execution)

stack pointer address of top of stack

Based on slides from Dan Page

ISA state: memory

Definition: Address space

The *address space* defines the range of accessible addresses in memory.

We also need to define further details about access to the address space:

addressability How many bits do we need to access elements of memory?

granularity What does each address refer to? E.g., byte-addressible if each element of the address space is a byte (8-bit word).

alignment For n -byte word at address x , if $x \equiv 0 \pmod n$ then x is aligned. Is unaligned access allowed?

endianness For an n -byte word, what order do we store the bytes in each word. E.g., $w = \langle w_0, w_1, \dots, w_{n-1} \rangle$

little-endian “left-to-right”, with the first bytes stored at lower memory addresses.

big-endian “right-to-left”, with the last bytes stored at lower memory addresses.

Endianness

The terms “big-endian” and “little-endian” came from Gulliver’s Travels, a 1726 book, in which citizens argued about which end of the egg they broke their boiled eggs.

See https://www.ling.upenn.edu/courses/Spring_2003/ling538/Lecnotes/ADfn1.htm and <https://en.wikipedia.org/wiki/Endianness>

ISA instructions

Instructions describe the operations the processor performs and how they change the state.

- ▶ Human-readable form, e.g., `add r0, r1, r2`
- ▶ Machine-readable form, e.g., `11100000100000010000000000000010`
- ▶ Semantic definition, e.g., $\text{GPR}[0] \leftarrow \text{GPR}[1] + \text{GPR}[2]$

The ISA needs to define how to encode/decode instructions, and what each instruction does.

Based on slides from Dan Page

ISA instructions: structure

opcode instruction operation (what it does)

operand(s) data to operate on

Operands can be *immediate* values, registers, memory addresses.

Opcode ensures the instruction is not ambiguous (e.g., ensures it is known whether in `add 1 2 3`, operands are values, registers, or addresses).

Component of ISA design is to determine what operands are allowed for each operation.

Usual to group instructions into *classes* with shared functionality: arithmetic, logic, comparison, memory access, control-flow, etc.

Based on slides from Dan Page

ISA instructions: encoding

How many bits do we use to represent each instruction?

fixed length encoding each instruction has the same length

variable length encoding each instruction has its own length

Fixed length encoding is easier to fetch and decode, but inefficient code density and more encoding pressure.

Based on slides from Dan Page

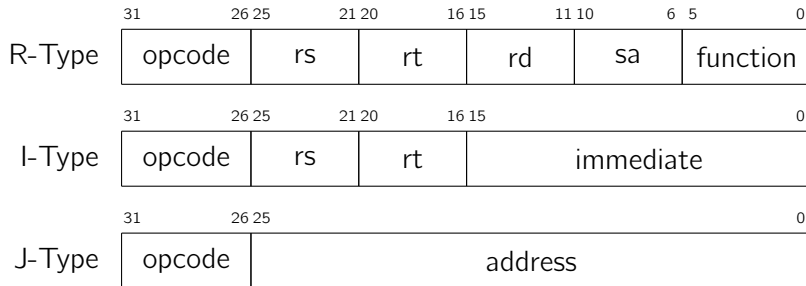
ISA instructions: encoding

And what do those bits represent?

Need to assign a number to each part of the instruction: opcode and each operand.

Also what order to represent them in the encoding, and how many bits to each part.

Example: MIPS32 (fixed length, 32-bits):



Based on slides from Dan Page

ISA instructions: encoding

And what do those bits represent?

Example: IA-32 (variable length (1–15 bytes))

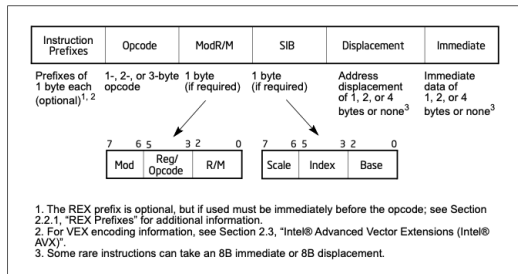


Figure 2-1. Intel 64 and IA-32 Architectures Instruction Format

From Intel 64 and IA-32 Architectures Software Developer's Manual, Vol 2A 2-1.,

<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

ISA types

Memory-memory architecture

All operands held in memory.

Register-memory architecture

Operands can be mixture of registers and memory.

Principal example is the Intel x86 ISA.

Register-register (load-store) architecture

Operands are registers. ISA has instructions to move data between registers and memory.

Principal examples are Arm, RISC-V, MIPS, ...

Based on slides from Dan Page

ISA types

Reduced Instruction Set Computing (RISC)

Typically register-register architecture with simple building-block instructions only.
Usually need good compilers to map high level software to lots of simple instructions.

Complex Instruction Set Computing (CISC)

Lots of instructions that do specific, multifaceted things so easy to map software to hardware.

Typically do lots of things all in one go: load data from memory, operate, store to memory.

Most CISC processors today are really RISC-like under-the-hood.

See <https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/riscisc/> and https://en.wikipedia.org/wiki/Complex_instruction_set_computer Based on slides from Dan Page

Instructions

For an instruction:

- ▶ The *architecture* (ISA) says what the instruction should do at a high level, e.g add two numbers together.
- ▶ The *micro-architecture* is how the processor implements each instruction.

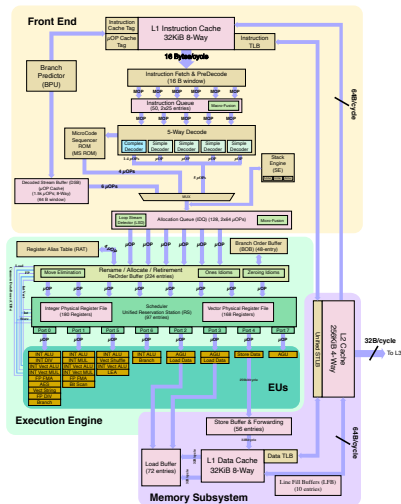
The micro-architecture design is free to choose how to implement, as long as it performs as specified by the ISA.

- ▶ An ISA can be implemented by different micro-architectures.
- ▶ Micro-architecture can execute any program written in the ISA.
- ▶ Processor vendors manufacture the micro-architecture.

Block diagrams

Common to use block diagrams to show micro-architecture design.

- ▶ Here is the block diagram for Intel Skylake processors.
- ▶ Shows design of one core.
- ▶ Front-end contains instruction fetch-decode.
- ▶ Execution Engine shows registers and functional units.
- ▶ Memory Subsystem to support access to memory.



Picture from https://en.wikipedia.org/w/images/7/7e/skylake_block_diagram.svg

Micro-architecture constraints

Micro-architectures designed under a number of constraints:

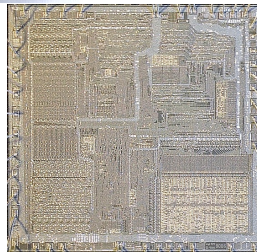
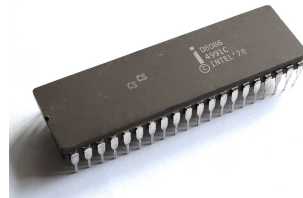
- ▶ Costs, area and production yield
- ▶ Performance
- ▶ Power use/thermal limit
- ▶ Reliability, security, etc.
- ▶ Many more . . .

Example: Intel and x86

x86 ISAs first developed for 16-bit Intel 8086 microprocessor (1978).

- ▶ Complex instruction set computer (CISC) design, with variable length instructions.
- ▶ Extended with SSE, to support a floating-point vector operations.

AMD also produce implementations of x86, along with x86_64, the 64-bit extension to x64.



Picture from https://en.wikipedia.org/wiki/Intel_8086#/media/File:Intel_D8086_CS.jpg, https://upload.wikimedia.org/wikipedia/commons/a/a8/Intel_8086_CPU_Die.JPG

Example: Intel, AMD and x86

Intel has produced many micro-architectures:

- ▶ 8086 (1978)
- ▶ i386 (1985) — 32-bit processor
- ▶ P5 Pentium (1993)
- ▶ ...
- ▶ Cascade Lake (2019)

All of these are x86.

They are all implementations of the ISA (although some add extensions).

Each generation typically:

- ▶ uses smaller lithography process.
- ▶ bring performance improvements though micro-architecture design changes.

Example: Arm

Arm ISA:

- ▶ Arm do not manufacture chips, but license the ISA (under various models) to others.
- ▶ Also provide a reference implementation (a micro-architecture). E.g. Neoverse.

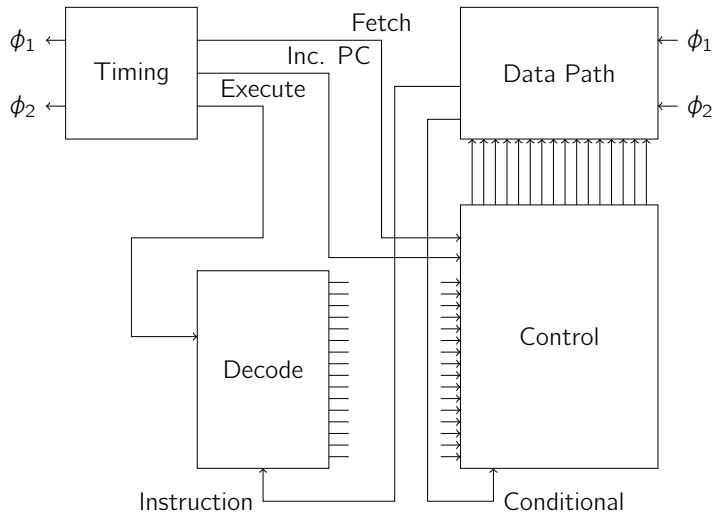


Pictures from: https://en.wikipedia.org/wiki/Apple-designed_processors#/media/File:Apple_T2_APL1027.jpg, <https://www.raspberrypi.org/products/compute-module-3-plus/>,

<https://www.anandtech.com/show/15578/cloud-clash-amazon-graviton2-arm-against-intel-and-amd/10>

Hex 8: Data path and Decode

Hex 8 micro-architecture: Structure



Data Path

Data Path

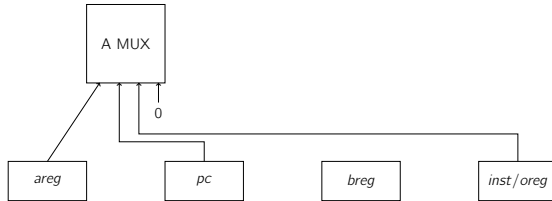
areg

pc

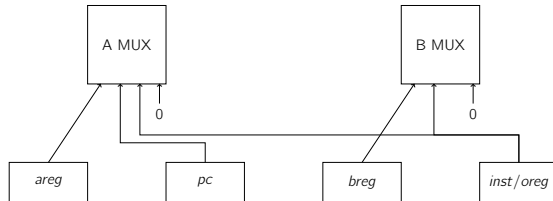
breg

inst/oreg

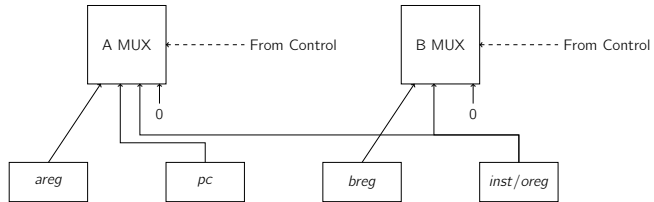
Data Path



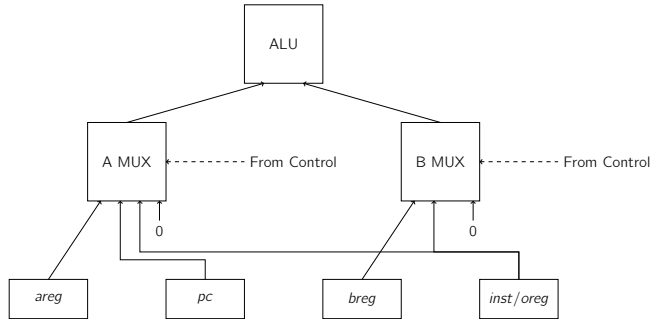
Data Path



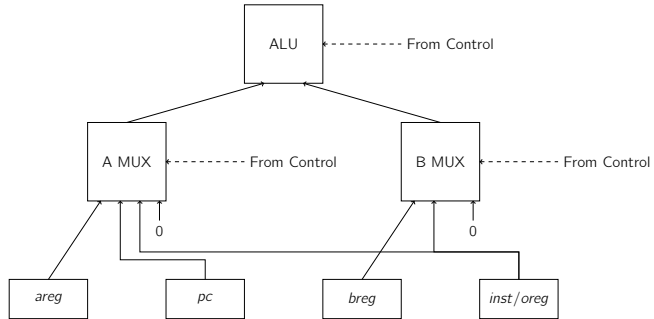
Data Path



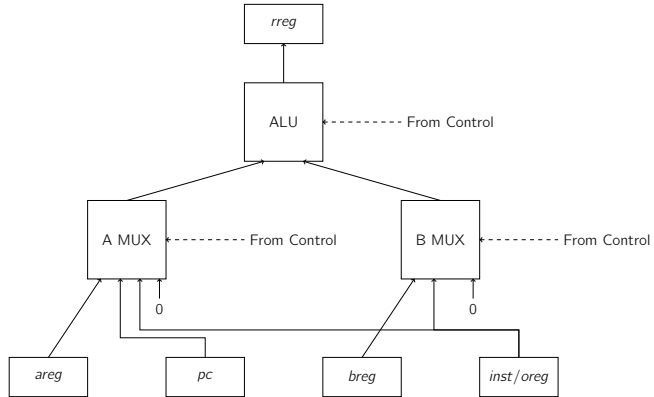
Data Path



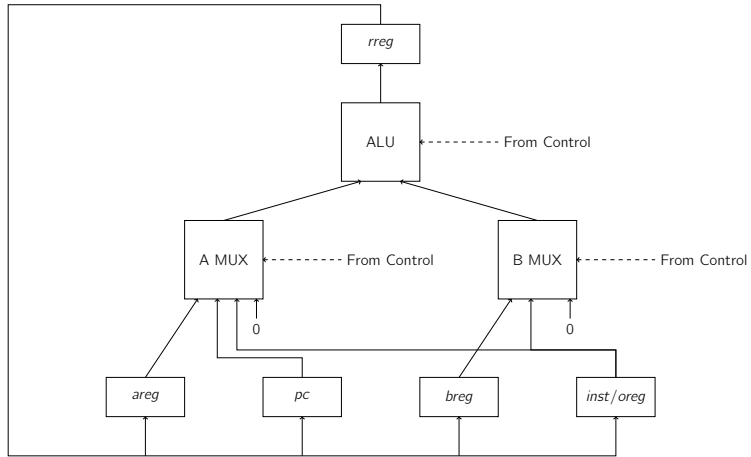
Data Path



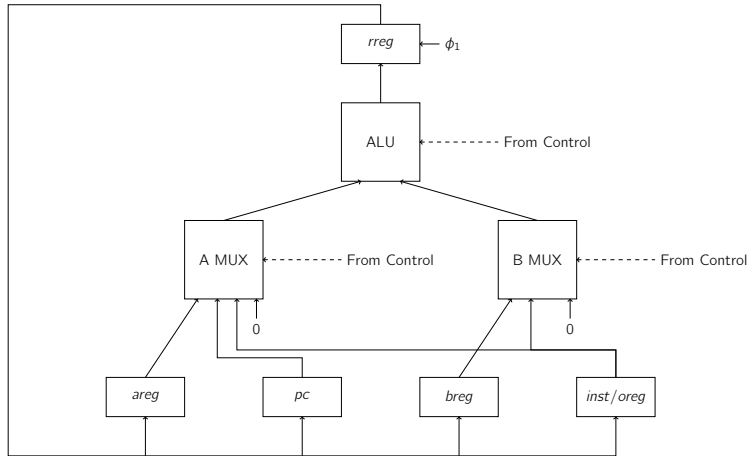
Data Path



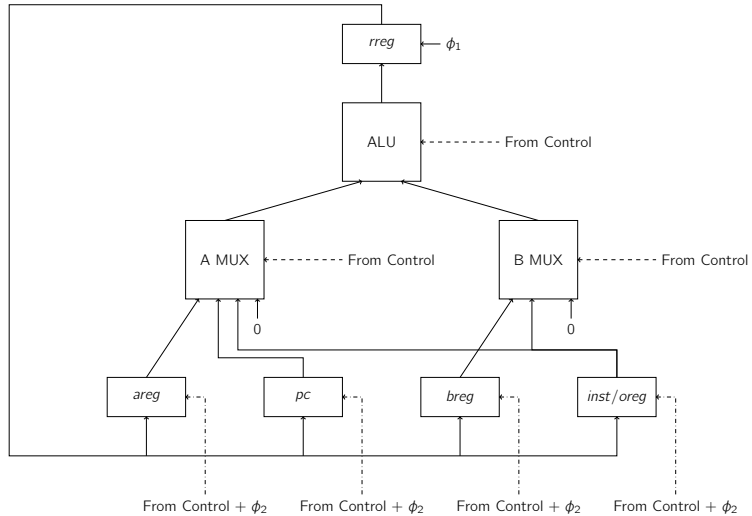
Data Path



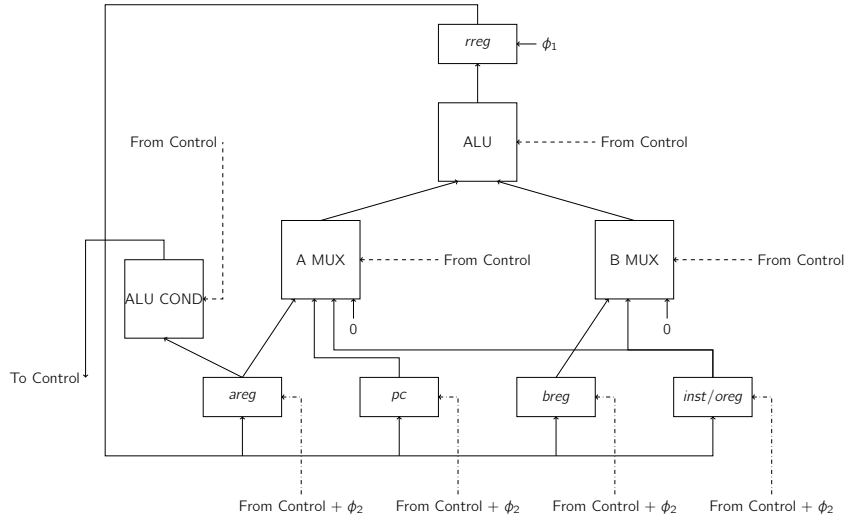
Data Path



Data Path



Data Path



Data Path

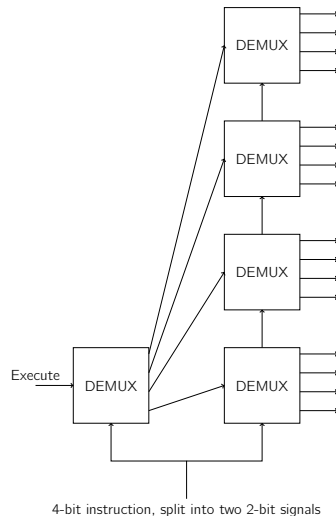
- ▶ Data path contains the registers and functional blocks to compute.
- ▶ Multiplexers used to select input registers to ALU.
- ▶ Wires come from Control to choose operations.
- ▶ Result register hidden from ISA — implementation detail.
- ▶ Two phase clock:
 - ϕ_1 : update *rreg* based on signals from Control.
 - ϕ_2 : save result in *areg/breg/oreg/pc*, also based on Control.

Note, Data Path as shown is missing the Memory and RMUX, which will be introduced later

Decode

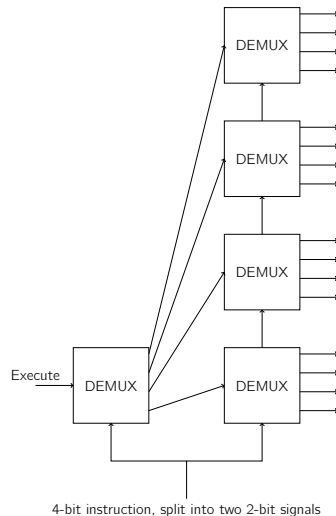
Decode

- ▶ Use DMUX to decode instruction.
- ▶ Split 4-bit instruction from *inst* register into two.
- ▶ Each 2-bit pair controls DMUX selection.



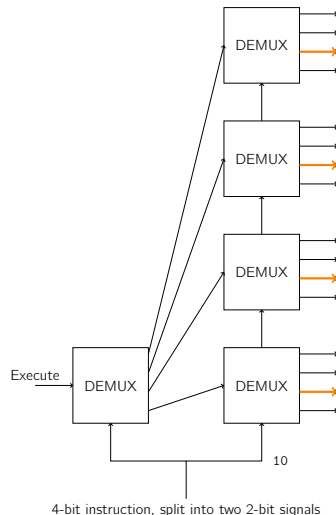
Decode

- ▶ Use DMUX to decode instruction.
- ▶ Split 4-bit instruction from *inst* register into two.
- ▶ Each 2-bit pair controls DMUX selection.
- ▶ Example: Instruction 0110 $\rightarrow$ 01 10



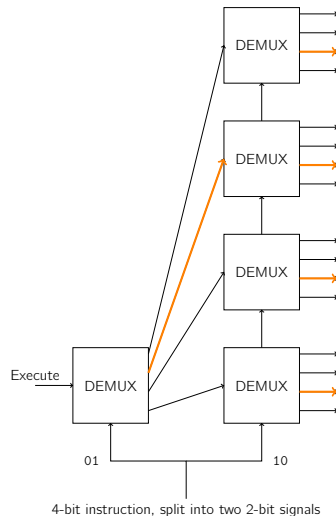
Decode

- ▶ Use DMUX to decode instruction.
- ▶ Split 4-bit instruction from *inst* register into two.
- ▶ Each 2-bit pair controls DMUX selection.
- ▶ Example: Instruction 0110 $\rightarrow$ 01 10
 - ▶ 10 selects third from top output on right DMUXs.



Decode

- ▶ Use DMUX to decode instruction.
- ▶ Split 4-bit instruction from *inst* register into two.
- ▶ Each 2-bit pair controls DMUX selection.
- ▶ Example: Instruction 0110 $\rightarrow$ 01 10
 - ▶ 10 selects third from top output on right DMUXs.
 - ▶ 01 selects second from top output on left DMUX.



Decode

- ▶ Use DMUX to decode instruction.
- ▶ Split 4-bit instruction from *inst* register into two.
- ▶ Each 2-bit pair controls DMUX selection.
- ▶ Example: Instruction 0110 $\rightarrow$ 01 10
 - ▶ 10 selects third from top output on right DMUXs.
 - ▶ 01 selects second from top output on left DMUX.
 - ▶ On Execute signal, left DMUX selects which right DMUX to activate — only one output signal.

